



김보현

Backend Developer.

대용량 트래픽과 데이터 정합성을 제어하며, 운영까지 고려한 아키텍처를 설계하는 개발자입니다.

기술의 동작 원리를 끝까지 파고들어 문제를 발견하고 상황에 맞는 아키텍처를 설계합니다.

거래 시스템을 구현하며 고객 보호를 위한 이상 거래 탐지, 운영팀을 위한 실시간 알림, 장애 추적을 위한 로그 시스템까지 직접 설계하고 구현했습니다.

About



2001.04.02



dkrmdckrmd@gmail.com



<https://github.com/dkrmdckrmd>



<https://baby-t.tistory.com/>

Skills



Java, Spring Boot



MySQL, JPA, Redis, MongoDB



Apache Kafka, Claude Code



Git, Spring Rest Docs



실시간 가상 자산 거래 플랫폼(개인 프로젝트)

2025.12 ~ 현재

프로젝트 링크 (클릭):

📖 프로젝트 개요

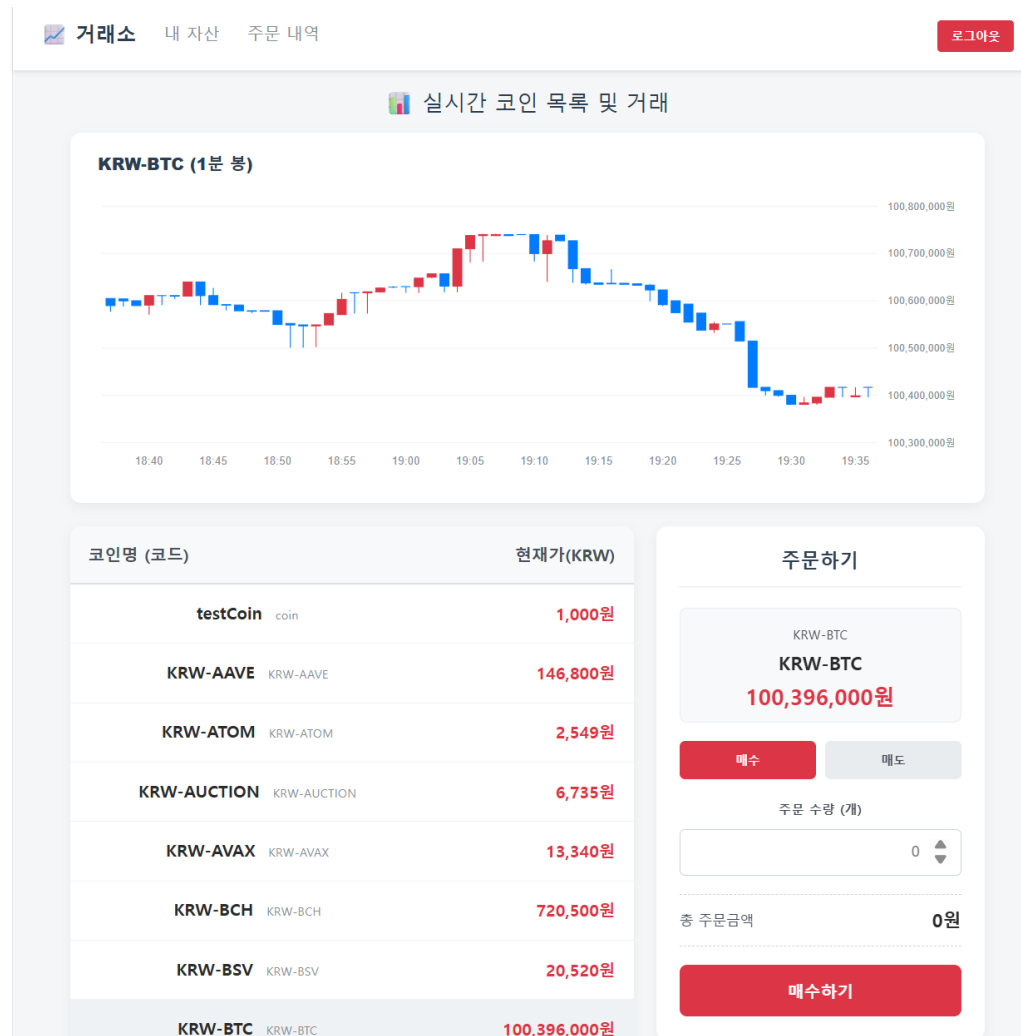
핵심 기능 : Upbit 연동 실시간 시세, 모의 투자, 수익률 집계

핵심 성과 :

- Redis Sorted Set 기반 이상 거래 탐지 (과다주문 자동 차단 + 4가지 정책 탐지)
- Kafka 비동기 처리로 응답속도 6.7초 -> 0.5초, TPS 1,500+, 에러율 0% 달성
- JWT Access + Refresh Token Rotation 구현 및 HttpOnly 쿠키 보안 강화
- MySQL + MongoDB 분리 설계 - 비즈니스 / 로그 데이터 분리
- Slack WebClient 논블로킹 비동기 실시간 알림 구축
- Oracle Cloud 인프라 이전 (RAM 1GB -> 24GB) 및 GitHub Actions CI / CD
- Nginx Reverse Proxy + SSL, Vercel 프론트 배포

🔧 사용한 기술

- ✓ Language / Framework : Java, Spring Boot, React
- ✓ Database / ORM : MySQL, MongoDB, JPA
- ✓ Cache / Messaging : Redis, Apache Kafka
- ✓ Security : Spring Security
- ✓ Testing / Docs : JUnit5, Mockito, JMeter, Spring REST Docs
- ✓ Infra / Tools : Git, Docker, Oracle Cloud, GitHub Actions, Claude Code

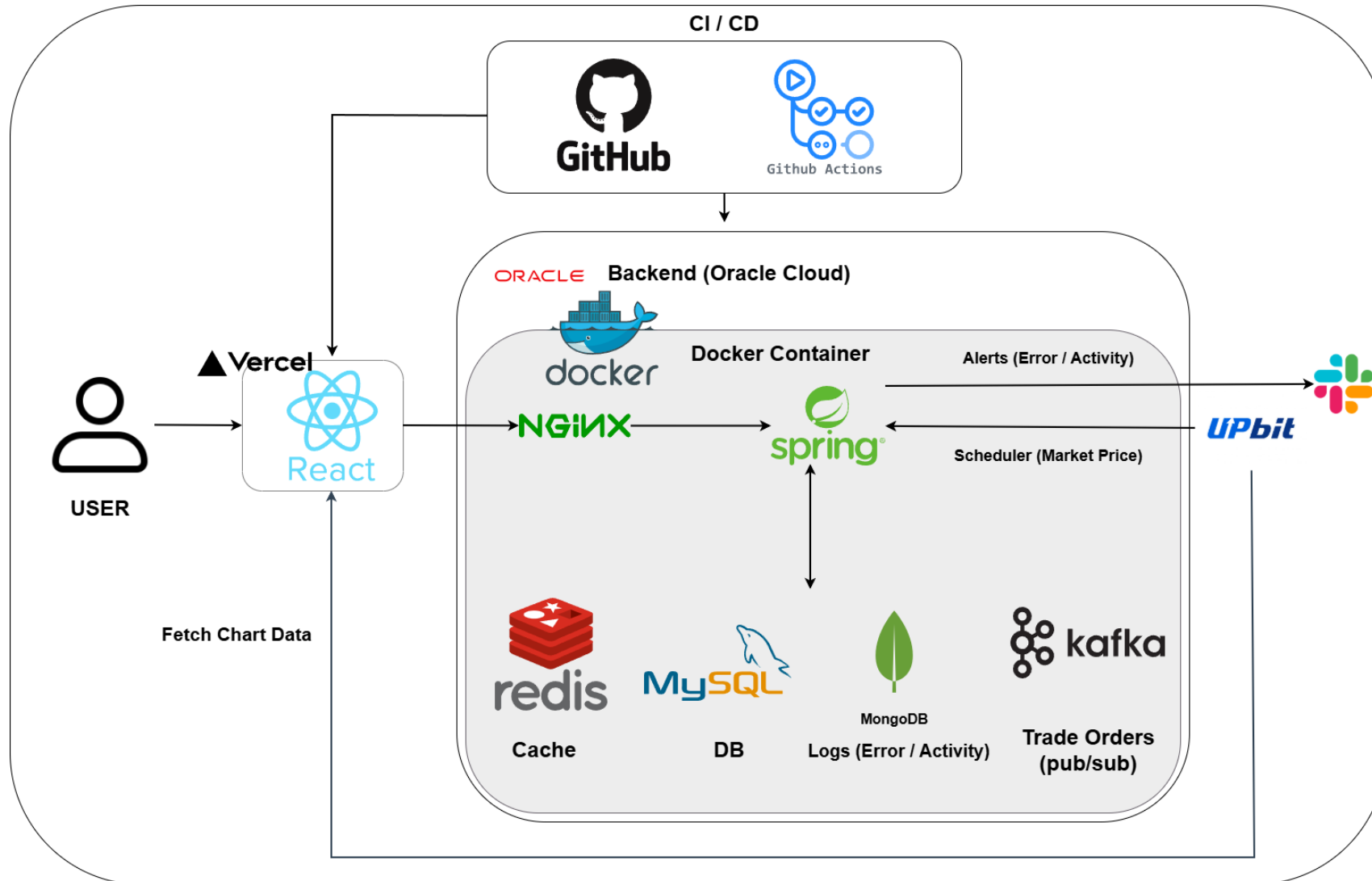




아키텍처 구조

2025.12 ~ 현재

깃허브 링크 (클릭):



이상 거래 탐지 시스템

 **Redis Sorted Set 기반 Sliding Window**로 실시간 이상 거래를 탐지하고 자동 차단하는 시스템 구축 (과다주문)

정책	조건	동작
과다 주문 차단	1분 내 5건 초과	주문 차단 + FAILED 저장 + 이메일 발송
대량 거래 감지	1,000 만원 이상	Slack 알림 + 이메일 발송
잔고 비율 감지	잔고 80% 이상 매수	Slack 알림 + 이메일 발송
야간 거래 감지	새벽 1~5시 + 잔고 50% 이상	Slack 알림 + 이메일 발송

```
// 단일 과다 주문
public void checkOverTrade(Long userId) { 2개 사용 위치 ⚡ dkrmddkrmd
    String key = ORDER_COUNT_PREFIX + userId;
    long now = System.currentTimeMillis();
    long windowStart = now - 60_000; // 60초 전

    // 1. 60초 이전 데이터 삭제
    redisTemplate.opsForZSet().removeRangeByScore(key, min: 0, windowStart);

    // 2. 현재 윈도우 내 주문 수 조회
    Long count = redisTemplate.opsForZSet().size(key);

    // 3. 5건 이상이면 차단
    if (count != null && count >= MAX_ORDER_PER_MINUTE) {
        slackNotificationService.sendAbnormalTradeBlocked(userId, reason: "1분 내 " + MAX_ORDER_PER_MINUTE + "회 초과 주문");
        throw new AbnormalTradeException("단시간 과다 주문으로 차단");
    }

    // 4. 현재 주문 timestamp 추가
    redisTemplate.opsForZSet().add(key, String.valueOf(now), now);

    // 5. 키 TTL 갱신 (60초)
    redisTemplate.expire(key, timeout: 60, java.util.concurrent.TimeUnit.SECONDS);
}
```

📄 주문 내역						
주문 일시	구분	종목	주문 수량	체결 가격	총 금액	상태
2026-04-26 06:36	매수	KRW-ETH	1	3,465,000원	3,465,000원	실패 단시간 과다 주문으로 차단
2026-04-26 06:36	매수	KRW-ETH	1	3,465,000원	3,465,000원	성공
2026-04-26 06:36	매수	KRW-ETH	1	3,465,000원	3,465,000원	성공
2026-04-26 06:36	매수	KRW-ETH	1	3,465,000원	3,465,000원	성공
2026-04-26 06:36	매수	KRW-ETH	1	3,465,000원	3,465,000원	성공
2026-04-26 06:36	매수	KRW-ETH	1	3,464,000원	3,464,000원	성공

[가상 거래소] 이상 거래 감지 알림 받은편지함 ✕



dkrmddkrmd@gmail.com

나에게 ▼

고객 거래가 감지되었습니다.

거래 정보: 1천만원 이상의 거래가 발생했습니다.

본인이 요청한 거래가 아니라면 즉시 비밀번호를 변경해주세요.

💡 차단 시 RuntimeException 발생으로 FAILED 주문이 롤백되는 문제 -> @Transactional(noRollbackFor) 적용으로 해결

운영 모니터링 시스템 (클로드 코드 활용)

장애 추적과 실시간 알리를 위한 MongoDB 로그 시스템 + Slack 알림 구축

MongoDB 로그 시스템

@LogActivity 커스텀 어노테이션 + AOP로 API 호출 자동 수집

@Async로 로그 저장 비동기 처리 -> API 응답 지연 없음

TTL 인덱스 30일 자동 만료 -> 저장소 자동 관리



트러블 슈팅

ActivityLogAspect 에서 직접 DB 접근 시 AOP 프록시 우회 발생



ActivityLogService 빈을 주입받아 호출하는 방식으로 변경

Slack 알림

WebClient.subscribe() 논블로킹 비동기 처리

Fail-safe 설계 -> Slack 장애 시에도 메인 로직 영향 없음

실제 Slack 화면



```
@Aspect  @dkrddkrmd
@Component
@RequiredArgsConstructor
public class ActivityLogAspect {

    private final ActivityLogService activityLogService;

    @AfterReturning("@annotation(logActivity)")  @dkrddkrmd
    public void logActivity(LogActivity logActivity) {
        Long userId = extractUserId();
        String ip = extractIp();
        String description = logActivity.description().isEmpty()
            ? logActivity.activityType().name()
            : logActivity.description();

        activityLogService.saveActivityLog(userId, logActivity.activityType(), description, ip);
    }
}
```

☆ # 운영-알림

● 메시지 🔄 캔버스 추가 +

🔴 이상 거래 차단

시간: 2026-04-25 21:32:12
UserID: 2
사유: 1분 내 5회 초과 주문

🔴 incoming-webhook 오후 11:16

🔴 서버 오류 발생!
시간: 2026-04-25 23:16:12
예외: KafkaProducerException
메시지: 카프카 서버로 주문을 전송하는 데 실패했습니다.

🔴 서버 오류 발생!

시간: 2026-04-25 23:16:33
예외: KafkaProducerException
메시지: 카프카 서버로 주문을 전송하는 데 실패했습니다.

🔴 서버 오류 발생!

시간: 2026-04-25 23:16:46
예외: KafkaProducerException
메시지: 카프카 서버로 주문을 전송하는 데 실패했습니다.

🔴 incoming-webhook 오후 11:28

🔴 이상 거래 감지!
시간: 2026-04-25 23:28:58
대상: test@test.com
사유: 1천만원 이상의 거래가 발생했습니다.

대용량 트래픽 검증 및 예외 처리 표준화

대용량 트래픽 부하 테스트

JMeter를 활용하여 1,000명의 유저가 총 10,000건의 주문을 동시에 요청하는 테스트 진행

Kafka 비동기 처리와 함께 **Error 0%**, **초당 처리량 1,500건** 달성



# Samples	Average	Min	Max	Std. Dev.	Error %	Throughput	Received KB/sec	Sent KB/sec	Avg. Bytes
10000	411	2	4034	817.94	0.00%	1504.4/sec	540.66	384.77	368.0
10000	411	2	4034	817.94	0.00%	1504.4/sec	540.66	384.77	368.0

예외 처리 표준화

Try-catch 대신 **@RestControllerAdvice**를 도입하여 예외 처리를 중앙에서 관리

커스텀 예외를 정의하여, 여러 상황별로 명확한 HTTP 상태 코드 (409 Conflict 등)와 메시지 반환



```
@Slf4j
@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(DuplicateEmailException.class)
    public ResponseEntity<ErrorResponseDto> handleDuplicateEmail(DuplicateEmailException ex) {
        log.error("중복 가입 시도 감지: {}", ex.getMessage(), ex);
        ErrorCode errorCode = ErrorCode.DUPLICATE_EMAIL;

        return ResponseEntity
            .status(errorCode.getStatus())
            .body(ErrorResponseDto.of(errorCode, errorCode.getMessage()));
    }
}
```

SpringSecurity + Refresh Token 및 테스트 기반 API 문서화

JWT 인증/인가 도입

SpringSecurity 자체 세션 기반 보안을 제거하고, JWT 기반 필터를 추가하여 API 요청 시 토큰 검증 및 인가된 사용자는 SecurityContext에 저장

Access Token(30분) + Refresh Token(7일) 분리

Refresh Token -> Redis 저장 + Rotation 방식

HttpOnly 쿠키로 전달 -> XSS 방어

재사용 감지 시 전체 세션 즉시 무효화



Access Token 만료 시 재발급 되는 모습



테스트 기반 API 문서화

MockMvc와 Spring REST Docs를 연동하여, API 문서화

단위 테스트 성공 시에만 최신화되는 신뢰성 높은 API 명세서 작성.



가상자산 거래 플랫폼 API 문서

Version 0.0.1-SNAPSHOT

1. 유저 API (User)

1.1 회원가입

새로운 유저가 거래소에 가입합니다.

HTTP request

```
POST /api/users HTTP/1.1
Content-Type: application/json
Content-Length: 65
Host: localhost:8080

{"email":"apiTest@test.com","password":"1234","name":"apiTester"}
```

Request fields

Path	Type	Description
email	String	사용자 이메일
name	String	사용자 이름
password	String	비밀번호 (4자리 이상)

HTTP response

```
HTTP/1.1 201 Created
Content-Type: text/plain; charset=UTF-8
Content-Length: 20
```

회원가입 성공!

트러블 슈팅 1 - 동시성 제어와 비동기 처리, 그리고 기능 단순화

문제 상황

JUnit을 사용하여 데이터 정합성 테스트
한 유저가 100번 동시 주문 요청을 통해 기존
보유 잔액보다 9번 추가 매수 성공

1. 시도한 횟수 : 100번
2. 성공한 주문 수 : 10개
3. 보유 주식 총합 : 10개
4. 최종 잔액 : 0원

필요: 1L
실제 : 10L



분석

별도에 제어가 없는 API 주문들은 순서 상관
없이 동시에 진행된다는 것을 확인

동시성 제어 방법 후보 검토

@Synchronized -> 단일 서버에서만 동작, 분산 환경 불가

비관적 락 - DB 부하 증가로 비적합

행동

Redis의 분산락을 적용한 Facade 패턴을
도입하여 동시성 제어 처리



```
public void createOrder(Long userId, OrderRequestDto requestDto) { 6개 사용 위치 ㄹ dkrmdkrm +
    String lockKey = "lock:user:" + userId;
    RLock lock = redissonClient.getLock(lockKey);

    try {
        boolean available = lock.tryLock(10, 1, TimeUnit.SECONDS);

        if (!available) {
            log.warn("[Lock 실패] 유저 {} 락 획득 실패 - 트래픽 과부하", userId);
            throw new IllegalStateException("현재 주문량이 많아 처리가 지연되고 있습니다.");
        }

        testOrderService.buy(userId, requestDto);
    }
}
```

결과

테스트 다시 진행 후 이번엔 데이터 정합성이 지켜지는 것을 확인

동시성 제어를 통해 데이터 정합성을 확보



1. 시도한 횟수 : 100번
2. 성공한 주문 수 : 1개 (기대값: 1)
3. 보유 주식 총합 : 1개
4. 최종 잔액 : 0원



다음 페이지 이어서

트러블 슈팅 1 – 동시성 제어와 비동기 처리, 그리고 기능 단순화

테스트 대용량 트래픽에서 구현되는 상황이라는 별도의 테스트 진행
JMeter를 사용하여 1000명의 유저가 동시에 주문 요청을 했을 때 가정
이 때 더 복잡한 비즈니스 로직을 구현했다는 가정 하에 **2초의 대기시간을 추가**



```
try {  
    Thread.sleep( millis: 2000 );  
} catch (InterruptedException e) {  
    e.printStackTrace();  
}  
  
testOrderService.buy(userId, requestDto);
```

문제 상황

유저의 평균 대기 시간이 **6.7초**로 매우 긴 것을 확인



Before

# Samples	Average	Min	Max
1000	6733	2800	10478
1000	6733	2800	10478



After

# Samples	Average	Min	Max
1000	555	257	932
1000	555	257	932

분석

매수 비즈니스 로직을 **동기 처리**
방식으로 구현했기 때문에, 이런
트래픽이 급증하는 상황에선 병목
현상이 발생하게 됨

```
2026-02-09T22:53:24.674+09:00 INFO 25608 --- [virtual-exchange] [ntainer#0-1-C-1] c.e.v.kafka.consumer.StackOrderConsumer : [Consumer] 처리 시작 (Partition: 1, User: 77)  
2026-02-09T22:53:24.674+09:00 INFO 25608 --- [virtual-exchange] [ntainer#0-0-C-1] c.e.v.kafka.consumer.StackOrderConsumer : [Consumer] 처리 시작 (Partition: 0, User: 144)  
2026-02-09T22:53:24.674+09:00 INFO 25608 --- [virtual-exchange] [ntainer#0-2-C-1] c.e.v.kafka.consumer.StackOrderConsumer : [Consumer] 처리 시작 (Partition: 2, User: 91)
```

행동

카프카를 도입하여 **Producer**를 통해 메시지를 전달하고 유저는 주문
접수 알림을 받음 -> **Consumer**가 뒤에서 알아서 로직 처리

Partition과 **Concurrency**를 각각 3개씩 도입하여 요청을 3개의
줄로 분산 처리

결과

실제 평균 요청 시간은
0.5초로 12배 빠른 처리를 보여줌



기능 단순화

Kafka의 파티션 키를 활용하여 동일한 사용자 요청이 동일 파티션으로
전달되도록 설계하였고, 이를 통해 **파티션 내에서 순차 처리가 보장됨**을 확인.

그 결과 별도의 Redis 분산 락 없이도 데이터 정합성을 유지 가능하여 락을
제거하고 **Kafka 비동기 처리 구조로 전환 -> 기능 단순화**

문제 상황

분석

```
@ManyToOne(fetch = FetchType.LAZY)
@JoinColumn(name = "stock_code")
private Stock stock;
```

행동

결과

1
0

내 자산 현황 화면 (이전 버전)

The interface shows a 'Virtual Exchange' header with a balance of 2,667,932,540 원. Below this, there are three columns: '종목명' (Asset Name), '보유 수량' (Quantity), '평균 단가' (Average Price), '현재가' (Current Price), '평가 금액' (Current Value), '평가 손익' (Current P/L), and '수익률' (ROI). The assets listed are KRW-AEVE, KRW-AUTOM, KRW-AXION, KRW-AVAX, KRW-BCH, and KRW-BSV, each with its respective quantity, average price, current price, current value, current P/L, and ROI.

종목명	보유 수량	평균 단가	현재가	평가 금액	평가 손익	수익률
비트코인	16주	50,000원	194,625,000원	3,140,000,000원	1,673,200,000원	209150.00%
KRW-AEVE	2주	191,825원	169,300원	167,800원	-45,990원	-11.74%
KRW-AUTOM	1,293주	3,475원	2,910원	3,762,630원	-730,545원	-16.26%
KRW-AXION	1주	7,320원	7,165원	7,165원	-159원	-2.12%
KRW-AVAX	941주	17,890원	13,410원	12,637,630원	-4,196,860원	-24.93%
KRW-BCH	1주	874,500원	777,000원	777,000원	-97,500원	-11.15%
KRW-BSV	31주	26,560원	21,060원	652,860원	-170,500원	-26.71%

```

Hibernate:
select
    sh1_0.id,
    sh1_0.avg_price,
    sh1_0.quantity,
    sl_0.code,
    sl_0.current_price,
    sl_0.name,
    sh1_0.user_id
from
    stock_holding sh1_0
join
    stock sl_0
    on sl_0.code=sh1_0.stock_code
where
    sh1_0.user_id=?

```

끝까지 봐주셔서 감사합니다.